

Universidade do Minho
Escola de Engenharia

Cálculo de Programas

Trabalho Prático (2024/25)

Lic. em Engenharia Informática

Grupo G4

a104445 Alexandre Marques Miranda
a104437 Ana Sá Oliveira
a104179 Sara Azevedo Lopes

janeiro 2025

Preâmbulo

Em Cálculo de Programas pretende-se ensinar a programação de computadores como uma disciplina científica. Para isso parte-se de um repertório de *combinadores* que formam uma álgebra da programação e usam-se esses combinadores para construir programas *composicionalmente*, isto é, agregando programas já existentes.

Na sequência pedagógica dos planos de estudo dos cursos que têm esta disciplina, opta-se pela aplicação deste método à programação em Haskell (sem prejuízo da sua aplicação a outras linguagens funcionais). Assim, o presente trabalho prático coloca os alunos perante problemas concretos que deverão ser implementados em Haskell. Há ainda um outro objectivo: o de ensinar a documentar programas, a validá-los e a produzir textos técnico-científicos de qualidade.

Antes de abordarem os problemas propostos no trabalho, os grupos devem ler com atenção o anexo A onde encontrarão as instruções relativas ao *software* a instalar, etc.

Valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Problema 1

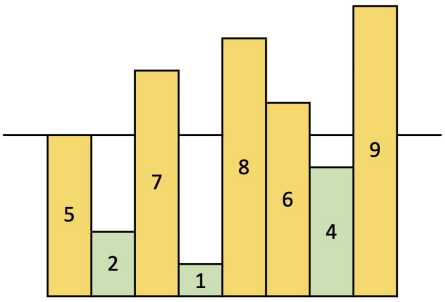
Esta questão aborda um problema que é conhecido pela designação '*H-index of a Histogram*' e que se formula facilmente:

> *O h-index de um histograma é o maior número n de barras do histograma cuja altura é maior ou igual a n.*

Por exemplo, o histograma

$$h = [5, 2, 7, 1, 8, 6, 4, 9]$$

que se mostra na figura



tem $hindex\ h = 5$ pois há 5 colunas maiores que 5. (Não é 6 pois maiores ou iguais que seis só há quatro.)

Pretende-se definida como um catamorfismo, anamorfismo ou hilomorfismo uma função em Haskell

$$hindex :: [Int] \to (Int, [Int])$$

tal que, para $(i, x) = hindex\ h$, i é o H-index de h e x é a lista de colunas de h que para ele contribuem.

A proposta de $hindex$ deverá vir acompanhada de um **diagrama** ilustrativo.

Problema 2

Pelo teorema fundamental da aritmética, todo número inteiro positivo tem uma única factorização prima. For exemplo,

```
primes 455
[5,7,13]
primes 433
[433]
primes 230
[2,5,23]
```

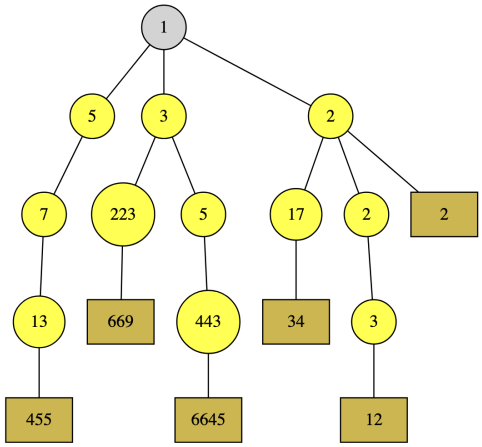
1. Implemente como anamorfismo de listas a função

   $$primes :: \mathbb{Z} \to [\mathbb{Z}]$$

   que deverá, recebendo um número inteiro positivo, devolver a respectiva lista de factores primos.

   A proposta de *primes* deverá vir acompanhada de um **diagrama** ilustrativo.

2. A figura mostra a "*árvore dos primos*" dos números $[455, 669, 6645, 34, 12, 2]$.



   Com base na alínea anterior, implemente uma função em Haskell que faça a geração de uma tal árvore a partir de uma lista de inteiros:

   $$prime_tree :: [\mathbb{Z}] \to Exp\ \mathbb{Z}\ \mathbb{Z}$$

   Sugestão: escreva o mínimo de código possível em *prime_tree* investigando cuidadosamente que funções disponíveis nas bibliotecas que são dadas podem ser reutilizadas.[1]

Problema 3

A convolução $a \star b$ de duas listas a e b — uma operação relevante em computação — está muito bem explicada neste vídeo do canal **3Blue1Brown** do YouTube, a partir de $t = 6:30$. Aí se mostra como, por exemplo:

[1] Pense sempre na sua produtividade quando está a programar — essa atitude será valorizada por qualquer empregador que vier a ter.

$$[1,2,3] \star [4,5,6] = [4,13,28,27,18]$$

A solução abaixo, proposta pelo chatGPT,

$$\begin{aligned}&convolve :: Num\ a \Rightarrow [a] \to [a] \to [a]\\&convolve\ xs\ ys = [sum\ \$\ zipWith\ (*)\ (take\ n\ (drop\ i\ xs))\ ys \mid i \leftarrow [0\,..\,(length\ xs - n)]]\\&\quad \textbf{where}\ n = length\ ys\end{aligned}$$

está manifestamente errada, pois $convolve\ [1,2,3]\ [4,5,6] = [32]$ (!).

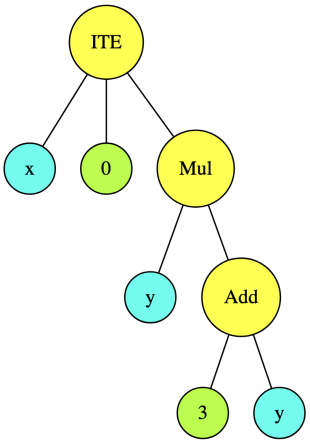
Proponha, explicando-a devidamente, uma solução sua para *convolve*. Valorizar-se-á a economia de código e o recurso aos combinadores *pointfree* estudados na disciplina, em particular a triologia *ana-cata-hilo* de tipos disponíveis nas bibliotecas dadas ou a definir.

Problema 4

Considere-se a seguinte sintaxe (abstrata e simplificada) para **expressões numéricas** (em b) com variáveis (em a),

$$\begin{aligned}&\textbf{data}\ Expr\ b\ a = V\ a \mid N\ b \mid T\ Op\ [Expr\ b\ a]\ \textbf{deriving}\ (Show, Eq)\\&\textbf{data}\ Op = ITE \mid Add \mid Mul \mid Suc\ \textbf{deriving}\ (Show, Eq)\end{aligned}$$

possivelmente condicionais (cf. *ITE*, i.e. o operador condicional "if-then-else"). Por exemplo, a árvore mostrada a seguir



representa a expressão

$$ite\ (V\ \texttt{"x"})\ (N\ 0)\ (multi\ (V\ \texttt{"y"})\ (soma\ (N\ 3)\ (V\ \texttt{"y"}))) \tag{1}$$

— i.e. $\textbf{if}\ x\ \textbf{then}\ 0\ \textbf{else}\ y * (3 + y)$ — assumindo as "helper functions":

$$\begin{aligned}&soma\ x\ y = T\ Add\ [x,y]\\&multi\ x\ y = T\ Mul\ [x,y]\\&ite\ x\ y\ z = T\ ITE\ [x,y,z]\end{aligned}$$

No anexo E propôe-se uma base para o tipo Expr (*baseExpr*) e a correspondente algebra *inExpr* para construção do tipo *Expr*.

1. Complete as restantes definições da biblioteca *Expr* pedidas no anexo F.
2. No mesmo anexo, declare *Expr b* como instância da classe *Monad*. **Sugestão**: relembre os exercícios da ficha 12.

3. Defina como um catamorfismo de *Expr* a sua versão monádia, que deverá ter o tipo:

$$mcataExpr :: Monad\ m \Rightarrow (a + (b + (Op, m\ [c])) \to m\ c) \to Expr\ b\ a \to m\ c$$

4. Para se avaliar uma expressão é preciso que todas as suas variáveis estejam instanciadas. Complete a definição da função

$$let_exp :: (Num\ c) \Rightarrow (a \to Expr\ c\ b) \to Expr\ c\ a \to Expr\ c\ b$$

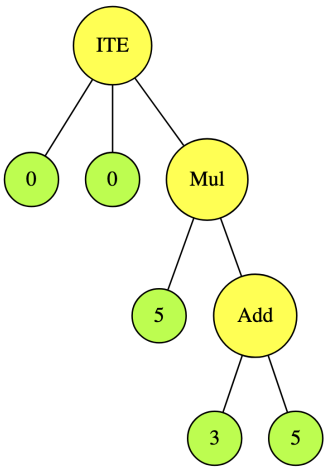
que, dada uma expressão com variáveis em a e uma função que a cada uma dessas variáveis atribui uma expressão ($a \to Expr\ c\ b$), faz a correspondente substituição.[1] Por exemplo, dada

$$f\ \texttt{"x"} = N\ 0$$
$$f\ \texttt{"y"} = N\ 5$$
$$f\ _ = N\ 99$$

ter-se-á

$$let_exp\ f\ e = T\ ITE\ [N\ 1, N\ 0, T\ Mul\ [N\ 5, T\ Add\ [N\ 3, N\ 1]]]$$

isto é, a árvore da figura a seguir:



5. Finalmente, defina a função de avaliação de uma expressão, com tipo

$$evaluate :: (Num\ a, Ord\ a) \Rightarrow Expr\ a\ b \to Maybe\ a$$

que deverá ter em conta as seguintes situações de erro:

(a) *Variáveis* — para ser avaliada, x em *evaluate* x não pode conter variáveis. Assim, por exemplo,

$$evaluate\ e = Nothing$$
$$evaluate\ (let_exp\ f\ e) = Just\ 40$$

para f e e dadas acima.

(b) *Aridades* — todas as ocorrências dos operadores deverão ter o devido número de subexpressões, por exemplo:

$$evaluate\ (T\ Add\ [N\ 2, N\ 3]) = Just\ 5$$
$$evaluate\ (T\ Mul\ [N\ 2]) = Nothing$$

[1] Cf. expressões **let** ... **in**....

Sugestão: de novo se insiste na escrita do mínimo de código possível, tirando partido da riqueza estrutural do tipo *Expr* que é assunto desta questão. Sugere-se também o recurso a diagramas para explicar as soluções propostas.

Anexos

A Natureza do trabalho a realizar

Este trabalho teórico-prático deve ser realizado por grupos de 3 alunos. Os detalhes da avaliação (datas para submissão do relatório e sua defesa oral) são os que forem publicados na página da disciplina na *internet*.

Recomenda-se uma abordagem participativa dos membros do grupo em **todos** os exercícios do trabalho, para assim poderem responder a qualquer questão colocada na *defesa oral* do relatório.

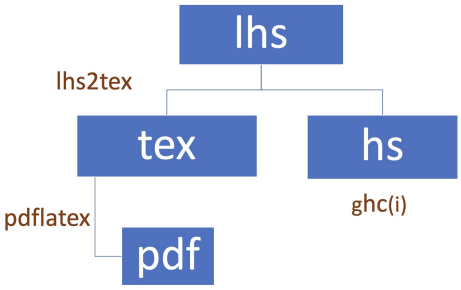
Para cumprir de forma integrada os objectivos do trabalho vamos recorrer a uma técnica de programação dita "literária" [**?**], cujo princípio base é o seguinte:

> *Um programa e a sua documentação devem coincidir.*

Por outras palavras, o **código fonte** e a **documentação** de um programa deverão estar no mesmo ficheiro.

O ficheiro `cp2425t.pdf` que está a ler é já um exemplo de programação literária: foi gerado a partir do texto fonte `cp2425t.lhs`[1] que encontrará no material pedagógico desta disciplina descompactando o ficheiro `cp2425t.zip`.

Como se mostra no esquema abaixo, de um único ficheiro (*lhs*) gera-se um PDF ou faz-se a interpretação do código Haskell que ele inclui:



Vê-se assim que, para além do GHCi, serão necessários os executáveis pdflatex e lhs2TeX. Para facilitar a instalação e evitar problemas de versões e conflitos com sistemas operativos, é recomendado o uso do Docker tal como a seguir se descreve.

B Docker

Recomenda-se o uso do container cuja imagem é gerada pelo Docker a partir do ficheiro `Dockerfile` que se encontra na diretoria que resulta de descompactar `cp2425t.zip`. Este container deverá ser usado na execução do GHCi e dos comandos relativos ao LaTeX. (Ver também a `Makefile` que é disponibilizada.)

[1] O sufixo 'lhs' quer dizer *literate Haskell*.

Após instalar o Docker e descarregar o referido zip com o código fonte do trabalho, basta executar os seguintes comandos:

```
$ docker build -t cp2425t .
$ docker run -v ${PWD}:/cp2425t -it cp2425t
```

NB: O objetivo é que o container seja usado *apenas* para executar o GHCi e os comandos relativos ao LaTeX. Deste modo, é criado um *volume* (cf. a opção `-v ${PWD}:/cp2425t`) que permite que a diretoria em que se encontra na sua máquina local e a diretoria `/cp2425t` no container sejam partilhadas.

Pretende-se então que visualize/edite os ficheiros na sua máquina local e que os compile no container, executando:

```
$ lhs2TeX cp2425t.lhs > cp2425t.tex
$ pdflatex cp2425t
```

lhs2TeX é o pre-processador que faz "pretty printing" de código Haskell em LaTeX e que faz parte já do container. Alternativamente, basta executar

```
$ make
```

para obter o mesmo efeito que acima.

Por outro lado, o mesmo ficheiro `cp2425t.lhs` é executável e contém o "kit" básico, escrito em Haskell, para realizar o trabalho. Basta executar

```
$ ghci cp2425t.lhs
```

Abra o ficheiro `cp2425t.lhs` no seu editor de texto preferido e verifique que assim é: todo o texto que se encontra dentro do ambiente

```
\begin{code}
...
\end{code}
```

é seleccionado pelo GHCi para ser executado.

C Em que consiste o TP

Em que consiste, então, o *relatório* a que se referiu acima? É a edição do texto que está a ser lido, preenchendo o anexo F com as respostas. O relatório deverá conter ainda a identificação dos membros do grupo de trabalho, no local respectivo da folha de rosto.

Para gerar o PDF integral do relatório deve-se ainda correr os comando seguintes, que actualizam a bibliografia (com BibTeX) e o índice remissivo (com `makeindex`),

```
$ bibtex cp2425t.aux
$ makeindex cp2425t.idx
```

e recompilar o texto como acima se indicou. (Como já se disse, pode fazê-lo correndo simplesmente `make` no container.)

No anexo E disponibiliza-se algum código Haskell relativo aos problemas que são colocados. Esse anexo deverá ser consultado e analisado à medida que isso for necessário.

Deve ser feito uso da programação literária para documentar bem o código que se desenvolver, em particular fazendo diagramas explicativos do que foi feito e tal como se explica no anexo D que se segue.

D Como exprimir cálculos e diagramas em LaTeX/lhs2TeX

Como primeiro exemplo, estudar o texto fonte (lhs) do que está a ler[1] onde se obtém o efeito seguinte:[2]

$$
\begin{array}{ll}
 & id = \langle f, g\rangle \\
\equiv & \quad \{\ \text{universal property}\ \} \\
 & \left\{ \begin{array}{l} \pi_1 \cdot id = f \\ \pi_2 \cdot id = g \end{array} \right. \\
\equiv & \quad \{\ \text{identity}\ \} \\
 & \left\{ \begin{array}{l} \pi_1 = f \\ \pi_2 = g \end{array} \right. \\
\Box &
\end{array}
$$

Os diagramas podem ser produzidos recorrendo à *package* xymatrix, por exemplo:

$$
\begin{array}{ccc}
\mathbb{N}_0 & \xleftarrow{\text{in}} & 1 + \mathbb{N}_0 \\
{\scriptstyle (\!|g|\!)}\downarrow & & \downarrow{\scriptstyle id + (\!|g|\!)} \\
B & \xleftarrow[g]{} & 1 + B
\end{array}
$$

E Código fornecido

Problema 1

$$h :: [Int]$$

Problema 4

Definição do tipo:

$$inExpr = [V, [N, \widehat{T}]]$$
$$baseExpr\ g\ h\ f = g + (h + id \times \text{map}\ \ f)$$

Exemplos de expressões:

$$e = ite\ (V\ \texttt{"x"})\ (N\ 0)\ (multi\ (V\ \texttt{"y"})\ (soma\ (N\ 3)\ (V\ \texttt{"y"})))$$
$$i = ite\ (V\ \texttt{"x"})\ (N\ 1)\ (multi\ (V\ \texttt{"y"})\ (soma\ (N\ (3\ /\ 5))\ (V\ \texttt{"y"})))$$

Exemplo de teste:

$$teste = evaluate\ (let_exp\ f\ i) \equiv Just\ (26\ /\ 245)$$
$$\textbf{where}\ f\ \texttt{"x"} = N\ 0; f\ \texttt{"y"} = N\ (1\ /\ 7)$$

[1] Procure e.g. por `"sec:diagramas"`.

[2] Exemplos tirados de [**?**].

F Soluções dos alunos

Os alunos devem colocar neste anexo as suas soluções para os exercícios propostos, de acordo com o "layout" que se fornece. Não podem ser alterados os nomes ou tipos das funções dadas, mas pode ser adicionado texto ao anexo, bem como diagramas e/ou outras funções auxiliares que sejam necessárias.

Importante: Não pode ser alterado o texto deste ficheiro fora deste anexo.

Problema 1

Para resolver o problema 1, do cálculo do H-index de um histograma e da lista de colunas do histograma que para ele contribuem, foi desenvolvida a seguinte solução:

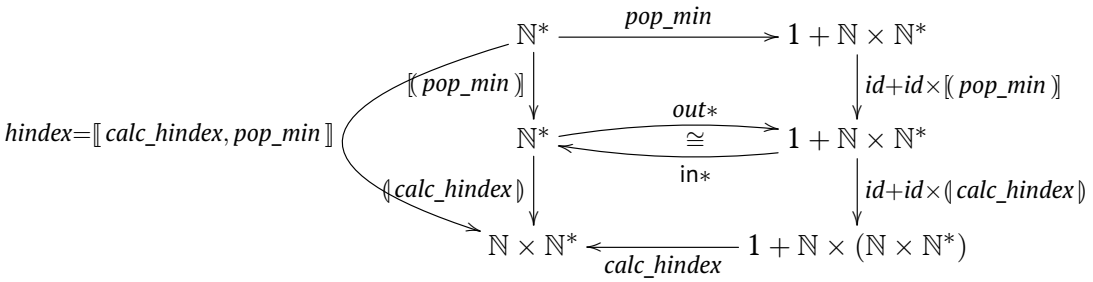
$$
\begin{aligned}
&\mathit{hindex} = \mathit{hyloList}\ \mathit{calc_hindex}\ \mathit{pop_min}\\
&\mathit{pop_min}\ [\,] = i_1\ ()\\
&\mathit{pop_min}\ \mathit{xs} = (i_2 \cdot \langle \pi_1, \widehat{\mathit{delete}} \rangle \cdot \langle \mathit{minimum}, \mathit{id} \rangle)\ \mathit{xs}\\
&\mathit{calc_hindex} = [\underline{0}, [\,], \cdot]\\
&\quad (\mathit{cond}\ (\widehat{(\geqslant)}) \cdot (\mathit{id} \times \mathsf{succ}\ \cdot \pi_1))\\
&\qquad \langle \mathsf{succ}\ \cdot \pi_1 \cdot \pi_2, \mathsf{conc} \cdot \mathit{swap} \cdot (\mathit{singl} \times \pi_2) \rangle\\
&\qquad \pi_2)
\end{aligned}
$$

A função `hindex` aplica um hilomorfismo de listas (`hyloList`) para determinar o H-index de uma lista que representa o histograma e para determinar a lista de colunas do histograma que contribuem para o H-index. Um hilomorfismo, como sabemos, é uma composição de um anamorfismo (que gera uma estrutura de dados ou outro tipo de estrutura a partir de um estado inicial) com um catamorfismo (que consome uma estrutura de dados, reduzindo-a a um único resultado). Assim, para construir o hilomorfismo, tivemos de descobrir as funções que são os genes do anamorfismo e do catamorfismo, respetivamente:

- `pop_min`: Esta função é o gene do anamorfismo que irá gerar uma estrutura de dados apartir do histograma. Este anamorfismo irá, mais concretamente, apartir do histograma, gerar uma lista ordenada por ordem crescente. Esta função calcula o mínimo de uma lista e a lista sem esse mínimo. Para uma lista vazia, devolve `i1()`, que corresponde a `Left ()`, encapsulando o valor da unidade `()` no lado esquerdo de um tipo `Either`. Para uma lista não vazia, a função identifica o valor mínimo da lista, remove a primeira ocorrência desse mínimo da lista e prepara o resultado num par, onde o primeiro elemento é o valor mínimo e o segundo é a lista com o mínimo removido. O resultado é então envolvido com `i2`, que é um sinónimo de `Right`, o lado direito de um tipo `Either`. Com este gene, o anamorfismo vai pegando no mínimo da lista e juntando à nova lista, o que irá gerar uma lista ordenada por ordem crescente. Este anamorfismo é o selection sort.
- `calc_hindex`: Esta função é o gene do catamorfismo que irá processar (consumir) a estrutura gerada pelo anamorfismo para calcular o H-index e determinar os elementos que contribuem para o resultado. O catamorfismo irá, mais concretamente, consumir a lista ordenada pelo anamorfismo, para obter o resultado que pretendemos. No gene, para um valor do tipo `Left` (`i1`), devolve diretamente o par `(0, [])`, representando que o H-index é zero e não há elementos processados. Para um valor do tipo `Right` (`i2`), avalia se o primeiro elemento do par (um valor da lista ordenada) é maior ou igual ao sucessor do primeiro elemento do segundo elemento do par (o H-index acumulado até ao momento). Se a condição for verdadeira, incrementa o contador (H-index) e adiciona o elemento atual à lista de contribuições. Caso contrário, devolve apenas o H-index anterior e a lista de contribuições anteriormente acumuladas. Com

este gene, o catamorfismo vai calculando o H-index e a lista que o justifica, iteração a iteração, para cada "sub-histograma", até por fim, calcular o H-index de todo o histograma e a lista que o justifica, aquilo que queriamos.

Diagrama do hilomorfismo:



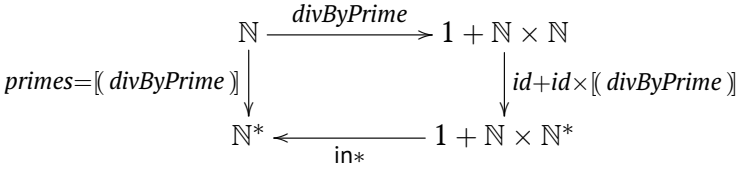
Problema 2

Primeira parte:

Para resolver o problema 2.1, de cálculo dos fatores primos de um número inteiro positivo, foi desenvolvida a seguinte solução:

$$
\begin{aligned}
&primes = [\!(\, divByPrime \,)\!] \\
&divByPrime\ 1 = i_1\ () \\
&divByPrime\ n = (i_2 \cdot \langle \pi_2, \widehat{\div} \cdot \rangle \cdot \langle id, fst_prime \rangle)\ n \\
&fst_prime = (head \cdot \widehat{filter} \\
&\quad \cdot \langle ((\equiv 0) \cdot) \cdot mod, \mathsf{conc} \\
&\quad \cdot \langle (2:) \cdot enumFromThenTo\ 3\ 5 \cdot floor \cdot sqrt \cdot fromIntegral, singl \rangle \rangle)
\end{aligned}
$$

Diagrama do anamorfismo:



Em primeiro lugar, definimos `primes` como sendo um anamorfismo de listas (`anaList`) porque, dado um número, esta função gera uma lista, logo temos aqui um anamorfismo de listas.
Depois temos de descobrir qual será o gene deste anamorfismo. A função `divByPrime` será o gene deste anamorfismo.

$$primes = [\!(\, divByPrime \,)\!]$$

Se a função receber o número 1, ela entrega `()` encapsulado em `i1`. Se a função receber outro número inteiro positivo, para além de 1, a função, apartir do número, calcula um par de valores (`split ou <,>`), sendo o primeiro valor o próprio número (`id`) e sendo o segundo valor o menor fator primo do número (`fst_prime`), o "primeiro" fator primo daquele número. Logo a seguir, apartir do par de valores obtido anteriormente (composição - `(.)`), calcula outro par de valores (`split ou <,>`), sendo o primeiro valor, o segundo valor do par anterior (`p2`), ou seja, o menor fator primo do número original e sendo o segundo valor a divisão inteira do par anterior (`uncurry div`), ou seja, a divisão do número original pelo seu menor fator primo. Apartir deste par de valores que obtivemos (composição - `(.)`), encapsulamos o par em `i2`.

$$
\begin{aligned}
&\mathit{divByPrime}\ 1 = i_1\ ()\\
&\mathit{divByPrime}\ n = (i_2 \cdot \langle \pi_2, \cdot \widehat{\div} \cdot \rangle \cdot \langle \mathit{id}, \mathit{fst_prime} \rangle)\ n
\end{aligned}
$$

Por fim, apenas nos falta definir o `fst_prime` usado na função anterior. Como já disse, ele calcula o menor fator primo do número dado. A primeira implementação do `fst_prime` foi a seguinte:

$$
\begin{aligned}
&\mathit{primes1} = [\!(\ \mathit{divByPrime1}\ )\!]\\
&\mathit{divByPrime1}\ 1 = i_1\ ()\\
&\mathit{divByPrime1}\ n = (i_2 \cdot \langle \pi_2, \cdot \widehat{\div} \cdot \rangle \cdot \langle \mathit{id}, \mathit{fst_prime1} \rangle)\ n\\
&\mathit{fst_prime1} = \mathit{head} \cdot \widehat{\mathit{filter}} \cdot \langle ((\equiv 0) \cdot) \cdot \mathit{mod}, \mathit{enumFromTo}\ 2 \rangle
\end{aligned}
$$

Aqui, apenas filtramos (`filter`) desde o número 2 (menor número primo) até ao nosso número (`enumFromTo 2`), aqueles que dividem o número inteiramente com resto 0 (`((==0) .) . mod`), e pegamos apenas no primeiro que cumpra este requisito (`head`) - o menor fator primo.

No entanto, apesar de simples, sabemos que esta solução é pouco eficiente, então decidimos implementar outra:

$$
\begin{aligned}
&\mathit{fst_prime} = (\mathit{head} \cdot \widehat{\mathit{filter}}\\
&\quad \cdot \langle ((\equiv 0) \cdot) \cdot \mathit{mod}, \mathrm{conc}\\
&\quad \cdot \langle (2:) \cdot \mathit{enumFromThenTo}\ 3\ 5 \cdot \mathit{floor} \cdot \mathit{sqrt} \cdot \mathit{fromIntegral}, \mathit{singl} \rangle \rangle)
\end{aligned}
$$

Nesta versão, a única diferença é a lista de onde iremos filtrar os números. Desta vez, obtemos uma lista menor, logo o cálculo para encontrar o menor fator primo será mais rápido, porque iremos filtrar de uma lista menor. Esta função, para além de mais eficiente, continua correta. É eficiente porque obtemos os números 2, o próprio número e os números impares de 3 até à raiz quadrada do nosso número, bem menos números do que anteriormente. Metade dos números que percorriamos (os números pares maiores que 2) agora não percorremos, porque sabemos que 2 é o único número par e primo, logo ao procurar fatores primos não vale a pena percorrer os outros números pares. Para além disso, já não percorremos os números da raiz quadrada do número até ao número (mas percorremos o número ainda), porque sabemos que para calcular os fatores primos de um número, basta procurar divisores primos até a raiz quadrada desse número (e também ver o próprio número). Isso ocorre porque, se o número tem um divisor maior que a sua raiz quadrada, então obrigatoriamente deve existir outro divisor menor que a raiz quadrada. Assim, ao percorrer apenas até a raiz quadrada, já encontramos todos os fatores primos (menos se o número for primo). Se até aqui não encontramos nenhum fator primo, é porque o número é primo, então no final da lista pomos o próprio número.
Por fim, decidimos ainda desenvolver uma versão que calcula os fatores primos mais eficientemente, em alguns casos:

$$
\begin{aligned}
&\mathit{primes_ef}\ \ n = \mathit{primes2}\ (2, n)\\
&\mathit{primes2} = [\!(\ \mathit{divByPrime2}\ )\!]\\
&\mathit{divByPrime2}\ (_, 1) = i_1\ ()\\
&\mathit{divByPrime2}\ (p, n) = (i_2 \cdot \langle \pi_2, \langle \pi_2, \cdot \widehat{\div} \cdot \rangle \rangle \cdot \langle \pi_2, \mathit{fst_prime2} \rangle)\ (p, n)\\
&\mathit{fst_prime2} = \mathit{head} \cdot \mathit{Cp.ap}\\
&\quad \cdot \langle \mathit{filter} \cdot ((\equiv 0) \cdot) \cdot \mathit{mod} \cdot \pi_2, \mathit{Cp.ap}\\
&\quad \cdot \langle \widehat{(\cdot)} \cdot ((:) \times \mathit{flip}\ (+\!\!+) \cdot \mathit{singl}), \mathit{Cp.ap}\\
&\quad \cdot (\mathit{Cp.ap} \cdot \langle \mathit{enumFromThenTo}, +2 \rangle \cdot \mathit{cond}\ (\equiv 2)\ (+1)\ (+2) \times \mathit{floor} \cdot \mathit{sqrt} \cdot \mathit{fromIntegral}) \rangle \rangle
\end{aligned}
$$

Diagrama do anamorfismo:

$$\begin{array}{ccc} \mathbb{N}\times\mathbb{N} & \xrightarrow{divByPrime2} & 1+\mathbb{N}\times(\mathbb{N}\times\mathbb{N}) \\ {\scriptstyle primes2=[\!(\, divByPrime2\,)\!]}\downarrow & & \downarrow{\scriptstyle id+id\times[\!(\, divByPrime2\,)\!]} \\ \mathbb{N}^* & \xleftarrow[\mathsf{in}*]{} & 1+\mathbb{N}\times\mathbb{N}^* \end{array}$$

Nesta versão, guardamos o menor fator primo da iteração anterior, para não tentarmos testar números que já deviamos saber que não são fatores primos deste número. Por exemplo, se anteriormente calculei que o menor fator primo de 45 é 3, na próxima iteração, saberei que o menor fator primo de 15 (3*15 = 45) é maior ou igual a 3. Já sei que não será o 2 porque senão teria sido o menor fator de 45 (nós pegamos nos fatores primos por ordem crescente). Assim, diminuimos a lista de procura de fatores primos e o programa fica mais eficiente.

Podemos perceber o quão mais eficiente esta função é da anterior, calculando os fatores primos de 99999971^3. Usando `primes`, primeiro ela vai percorrer a lista [2,3,5,7,...99999971] até encontrar o primeiro primo - 99999971. Depois voltará a percorrer [2,3,5,7,...99999971], etc...

Usando `primes_ef`, primeiro ela vai percorrer a lista [2,3,5,7,...99999971] até encontrar o primeiro primo - 99999971. Depois irá percorrer [99999971] porque já sabe que os números menores que 99999971 não são os fatores primos que faltam do número inicial. Claramente percebemos que a lista percorrida vai diminuir drasticamente em alguns casos, e assim, `primes_ef` é bem mais eficiente que `primes` em alguns casos. Noutros casos `primes` e `primes_ef` são igualmente eficientes (por exemplo, para calcular os fatores primos de 99999971, já vimos que percorrem a mesma lista).

Segunda parte:

Para resolver o problema 2.2, de gerar uma "árvore dos primos", do tipo `Exp v o`, dada uma lista de números inteiros positivos, foi desenvolvida a seguinte solução:

$$prime_tree = Term\ 1 \cdot untar \cdot \mathsf{map}\ \langle primes, id\rangle$$

Em primeiro lugar, percebemos que teriamos de primeiramente calcular os fatores primos de cada número da lista e que teriamos de manter os números da lista, porque na árvore tanto temos os fatores primos de cada número, como temos os próprios números da lista inicial (como podemos observar na árvore dada como exemplo). Assim, a função que queremos tem de começar por calcular, para todos os números da lista (função `map`), um par de valores apartir do número (função `split` ou <,>): o primeiro elemento do par é os seus fatores primos (função `primes`) e o segundo elemento é o próprio número (função `id`). Assim, em `prime_tree` começamos com a seguinte função:

$$\mathsf{map}\ \langle primes, id\rangle$$

Em segundo lugar, percebemos que qualquer árvore de primos terá o número 1, como sendo a raiz da árvore, tal como no exemplo que nos foi dado. Nem todos os números inteiros positivos têm fatores primos em comum, no entanto, todos os números inteiros positivos têm em comum o facto de serem divisíveis por 1, têm em comum o fator 1. O número 1 é um fator universal, pois divide qualquer número inteiro positivo, sempre resultando em quociente inteiro e resto 0. Por isso, faz sentido, na árvore de primos, o número 1 ser a raiz da árvore, o ponto de partida. No entanto, sabemos que o número 1 não é primo, então este não será nenhum fator primo obtido na função anterior. Então, o 1 terá de ser inserido na árvore. Como a árvore é do tipo `Exp v o`, para o número 1 ser a raiz da árvore, a árvore terá de ser `Term 1 o`, sendo o a lista de "subárvores", ou seja, `[Exp v o]`. Assim, em `prime_tree` terminamos com a seguinte função:

$$Term\ 1$$

Em terceiro lugar, percebemos que teríamos de usar uma função que organizasse os números e os seus fatores primos, que já obtemos, numa lista de subárvores (`[Exp v o]`) para podermos depois obter a tal árvore de primos desejada, com a raiz igual ao número 1. Ou seja, precisamos de uma função cuja entrada seja a saída da função `map <primes,id>`, ou seja, uma lista de pares, cujo o primeiro elemento é uma lista de inteiros (os fatores primos) e o segundo é um inteiro (o número), ou seja, do tipo `[([o],v)]`, e cuja saída seja a entrada da função `Term 1`, ou seja, do tipo `[Exp v o]` (a lista de subárvores). No fundo, queremos primeiro aplicar a função `map <primes,id>` depois aplicar a tal função desejada e por fim, aplicar a função `Term 1`. O tipo da função desejada será então `[([o], v)] -> [Exp v o]`.

$$Term\ 1 \cdot ? \cdot \mathsf{map}\ \langle primes, id\rangle$$
$$? :: [([o], v)] \to [Exp\ v\ o]$$

Como a saída da função que queremos é do tipo `[Exp v o]`, então decidimos investigar cuidadosamente a biblioteca Exp.hs, biblioteca dedicada ao tipo de dados `Exp v o`. Rapidamente, encontramos a função `untar`, que tem o mesmo tipo, que a função desejada.

$$untar :: (Ord\ v, Ord\ o) \Rightarrow [([o], v)] \to [Exp\ v\ o]$$

Testamos a nossa função `prime_tree` que usa `untar` e rapidamente percebemos que fazia aquilo que desejavamos. A função `untar`, no fundo, reorganiza os números e os seus fatores primos, que se encontravam num formato `[([o], v)]`, no formato `[Exp v o]`, respeitando uma certa ordem (`(Ord v, Ord o)`). Afinal, como conseguimos ver na árvore dada como exemplo, na árvore os fatores primos de um número vão aparecendo na árvore por ordem crescente até chegarmos ao número, que é uma folha. Com esta função conseguimos construir a função `prime_tree`:

$$prime_tree = Term\ 1 \cdot untar \cdot \mathsf{map}\ \langle primes, id\rangle$$

Assim, conseguimos resolver o problema 2.2 escrevendo o mínimo de código possível (1 linha). Isto foi conseguido porque reutilizámos uma função da biblioteca Exp.hs, a função `untar`, que nos poupou muito trabalho.

Problema 3

Para resolver o problema 3, do cálculo da convolução de duas listas a e b, foi desenvolvida a seguinte solução:

$$convolve :: Num\ a \Rightarrow [a] \to [a] \to [a]$$
$$convolve = ((Cp.ap \cdot \langle(\widehat{hyloList}\cdot)\ (\overline{combineWindows} \times (\overline{generateWindows} \cdot \pi_2)), pad \cdot \pi_2\rangle)\cdot)$$
$$\cdot \overline{\langle reverse \cdot \pi_2, id \times length\rangle}$$
$$pad :: Num\ a \Rightarrow ([a], Int) \to [a]$$
$$pad = \mathsf{conc} \cdot \langle \pi_2, \mathsf{conc}\rangle \cdot (id \times (\text{`}replicate\text{`}0) \cdot (subtract\ 1))$$
$$generateWindows :: Num\ a \Rightarrow (Int, [a]) \to () + ([a], [a])$$
$$generateWindows = cond\ (\widehat{(>)} \cdot (id \times length))\ \underline{i_1\ ()}\ (i_2 \cdot \langle\widehat{take}, tail \cdot \pi_2\rangle)$$
$$combineWindows :: Num\ a \Rightarrow ([a], () + ([a], [a])) \to [a]$$
$$combineWindows = Cp.ap \cdot ((\lambda ys \to [\underline{[\,]}, \mathsf{conc} \cdot \langle singl \cdot sum \cdot (zipWith\ (*)\ ys) \cdot \pi_1, \pi_2\rangle]) \times id)$$

A função `convolve` implementa a convolução discreta de duas listas, utilizando um hilomorfismo (`hyloList`) para gerar e combinar janelas de valores. Um hilomorfismo, como sabemos, é uma composição de um anamorfismo (que gera uma estrutura de dados, mais especificamente aqui irá gerar

janelas) com um catamorfismo (que consome uma estrutura de dados, reduzindo-a a um único resultado, mais especificamente aqui irá combinar janelas dando origem a um resultado). Assim, para construir o hilomorfismo, tivemos de descobrir as funções que são os genes do anamorfismo e do catamorfismo e ainda tivemos de desenvolver outras funções auxiliares. Antes de entrarmos no hilomorfismo, obtemos a primeira lista (`id`) e o comprimento da segunda lista (`length`). Depois obtemos também a segunda lista revertida (como se fazia no vídeo) (`(reverse . p2)`). Depois vamos usar a primeira lista e comprimento da segunda lista, já obtidos, para preparar uma lista adequada para ajudar a gerar e combinar janelas (a lista que irá ser usada no hilomorfismo). Depois associamos o comprimento da segunda lista ao anamorfismo e associamos a segunda lista revertida ao catamorfismo.

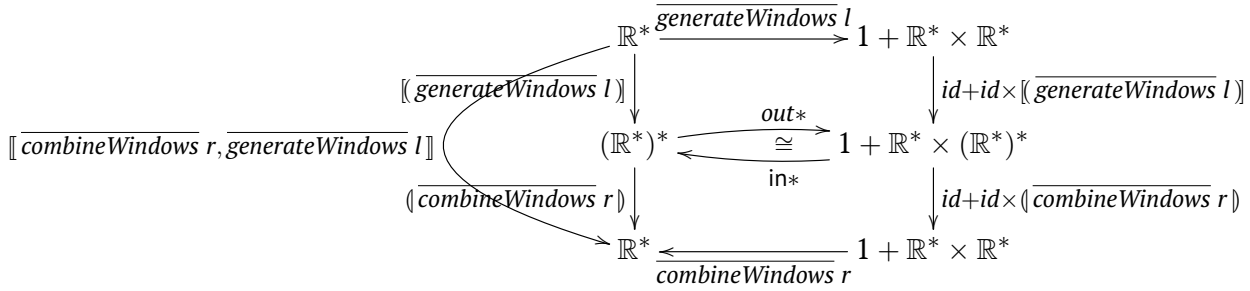
- `pad`: Uma função que adiciona o número adequado de zeros no final e no início da lista. Recebe um par onde o primeiro elemento é a primeira lista e o segundo é o comprimento da segunda lista. Apartir do par dado (><), mantém o primeiro elemento, mantém a primeira lista (`id`) e pega no segundo elemento, comprimento da segunda lista, retira-lhe um valor (`subtract 1`) e depois (`.`) replica o 0 esse número de vezes (`` `replicate` 0``), ou seja, forma uma lista com esse número de 0s. Apartir do par (`split ou <,>`), mantém a lista de zeros (`p2`) e concatena a primeira lista com a mesma lista de zeros (`conc`). Depois (`.`), pega no par que resta e concatena a lista de zeros com a lista anteriormente obtida (a lista que contém a primeira lista e zeros no final). E assim, obtemos uma lista com zeros no início e final da lista e que no meio tem a primeira lista original. Esta lista será usada no hilomorfismo. A função replica zeros e concatena-os à lista original.
- `generateWindows`: Esta função é o gene do anamorfismo que irá gerar as janelas. Esta função gera uma janela de valores da "lista com zeros". Para evitar casos inválidos, verifica se o comprimento da segunda lista original excede o comprimento da janela atual. Se for o caso, devolve `i1()`, representando que não há mais janelas. Caso contrário, devolve um par envolvido em `i2`, contendo a nova janela gerada (que geramos pegando nos primeiros x elementos da lista com zeros, sendo x o comprimento da segunda lista original) e a parte restante da "lista com zeros" sem a cabeça (`tail`).
- `combineWindows`: Esta função é o gene do catamorfismo que irá combinar as janelas, que no fundo irá aplicar a convolução. Esta função combina uma janela gerada com os valores da segunda lista revertida. Para cada janela, calcula o produto escalar entre a janela e os valores correspondentes da segunda lista revertida, somando os resultados. Depois faz uma lista com apenas esta soma e concatena esta lista, com a lista acumuladora de somas, que acumulou as somas de iterações anteriores. Se não houver janelas (`Left`), devolve uma lista vazia.

A função `convolve` aplica estas operações para calcular a convolução discreta, gerando uma nova lista onde cada elemento é o resultado da combinação das janelas com os valores da segunda lista revertida.

Antes de termos a função convolve, tinhamos a função convolve1. Ambas fazem o mesmo, mas a convolve1 é bem mais fácil de entender devido a ter variáveis. Por isso, mostramos agora a função convolve1, equivalente à função convolve, mas com variáveis:

$$
\begin{aligned}
&\mathit{convolve1} :: \mathit{Num}\ a \Rightarrow [a] \to [a] \to [a] \\
&\mathit{convolve1}\ \mathit{xs}\ \mathit{ys} = \mathit{hyloList}\ \overline{\mathit{combineWindows}}\ r\ \overline{\mathit{generateWindows}}\ l\ (\mathit{pad}\ (\mathit{xs}, l)) \\
&\quad \textbf{where} \\
&\qquad l = \mathit{length}\ \mathit{ys} \\
&\qquad r = \mathit{reverse}\ \mathit{ys}
\end{aligned}
$$

Diagrama do hilomorfismo presente em convolve1:



Problema 4

Para resolver o problema 4.1, tivemos de definir `outExpr`, `recExpr`, `cataExpr`, `anaExpr` e `hyloExpr`.

Definição do tipo:

Em primeiro lugar, definimos `outExpr`. Se `inExpr` converte um valor do tipo `Either a (Either b (Op, [Expr b a]))` para um valor do tipo `Expr b a`, então `outExpr` irá fazer o oposto, ou seja, `outExpr` converte um valor do tipo `Expr b a` para um valor do tipo `Either a (Either b (Op, [Expr b a]))`:

$$outExpr\ (V\ a) = i_1\ a$$
$$outExpr\ (N\ b) = (i_2 \cdot i_1)\ b$$
$$outExpr\ (T\ op\ l) = (i_2 \cdot i_2)\ (op, l)$$

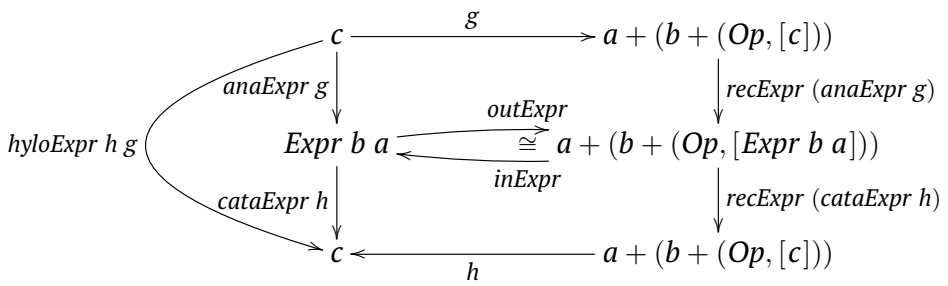
Em segundo lugar, definimos `recExpr`. A função `recExpr f` é o mesmo que `F f`, para o tipo de dados `Expr b a`. `F f = id + (id + id >< map f)` que é equivalente a:

$$recExpr = baseExpr\ id\ id$$

Ana + cata + hylo:

Em terceiro lugar, definimos `cataExpr`, `anaExpr` e `hyloExpr` e ainda fizemos um diagrama ilustrativo.

$$cataExpr\ h = h \cdot (recExpr\ (cataExpr\ h)) \cdot outExpr$$
$$anaExpr\ g = inExpr \cdot (recExpr\ (anaExpr\ g)) \cdot g$$
$$hyloExpr\ h\ g = cataExpr\ h \cdot anaExpr\ g$$



Monad:
Para resolver o problema 4.2, tivemos de declarar `Expr b` como instância da classe Monad.

```
instance Functor (Expr b) where
  fmap f = cataExpr (inExpr . baseExpr f id id)
instance Applicative (Expr b) where
  (<*>) = aap
  pure = return
instance Monad (Expr b) where
  return = V
  (V a) >>= g = g a
  (N b) >>= g = N b
  (T op l) >>= g = T op (map (>>=g) l)
```

Em primeiro lugar, declaramos `Expr b` como instância da classe Monad. Para declarar `Expr b` como instância da classe Monad, em Haskell, tivemos de implementar as funções `return` e `(>>=)`. O `return` em Haskell é a unidade. Com o `(>>=)` do Haskell podemos implementar a multiplicação, `mu`, como fizemos em baixo. Nós colocamos `return = V` porque `return` insere um valor no contexto monádico, e como V representa uma variável genérica em Expr, ele é o construtor apropriado para encapsular valores manipuláveis. Depois implementamos `(>>=)`. Para uma variável `(V a)`, o operador >>= aplica a função g ao valor encapsulado, retornando o resultado da transformação. Para um número `N b`, como números não devem ser modificados, o valor é retornado inalterado. Para uma operação `T op l`, >>= aplica recursivamente a função g a cada subexpressão na lista l, mantendo a estrutura da operação.

```
mu :: Monad m => m (m b) -> m b
mu x = x >>= id
```

Para o programa compilar, tivemos de declarar ainda `Expr b` como instância das classes Applicative e Functor. Todas as monades são primeiro funtores então faz sentido termos de declarar `Expr b` como instância de Functor. Para isso, tivemos de implementar a função `fmap`.

$$\begin{array}{ccc} Expr\ b\ a & \underset{inExpr}{\overset{outExpr}{\cong}} & a + (b + (Op, [Expr\ b\ a])) \\ \Big\downarrow{\scriptstyle \text{fmap } f = cataExpr\ (inExpr \cdot baseExpr\ f\ id\ id)} & & \Big\downarrow{\scriptstyle recExpr\ (\text{fmap } f)} \\ Expr\ b\ a & \xleftarrow[inExpr \cdot baseExpr\ f\ id\ id]{} & a + (b + (Op, [Expr\ b\ a])) \end{array}$$

Desde a introdução do "Applicative Monad Proposal" (AMP), a classe Monad agora exige que a classe Applicative seja implementada antes. Isso significa que também precisamos implementar pure e <*>, que vêm da classe Applicative e foi isso que fizemos.

Maps: *Monad*: *Let expressions*:

Para resolver o problema 4.4, usamos a função `(>>=)` anteriormente definida.

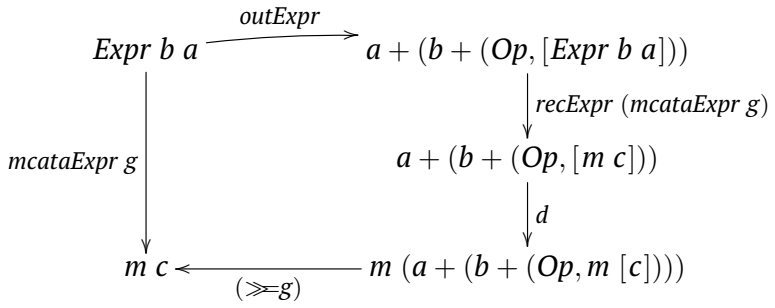
```
let_exp f = (>>=f)
```

A função `let_exp` aplica a função `f` a cada variável de uma expressão, substituindo-a pela expressão correspondente fornecida por `f`. Usando o operador >>=, a função substitui as variáveis dentro da expressão, garantindo que todas as variáveis sejam instanciadas antes de a expressão poder ser avaliada.

Catamorfismo monádico:

Para resolver o problema 4.3, de definir o catamorfismo monádico deste tipo de dados, desenvolvemos a seguinte solução:

$$
\begin{aligned}
&mcataExpr\ g = g\ .!\ (d \cdot recExpr\ (mcataExpr\ g) \cdot outExpr)\\
&d :: Monad\ m \Rightarrow a + (b + (Op, [m\ c])) \to m\ (a + (b + (Op, m\ [c])))\\
&d = [return \cdot i_1, [return \cdot i_2 \cdot i_1, \mathsf{fmap}\ (i_2 \cdot i_2) \cdot l]]\\
&l :: Monad\ m \Rightarrow (Op, [m\ c]) \to m\ (Op, m\ [c])\\
&l\ (op, t) = \mathbf{do}\\
&\quad mcs \leftarrow sequence\ t\\
&\quad return\ (op, return\ mcs)
\end{aligned}
$$



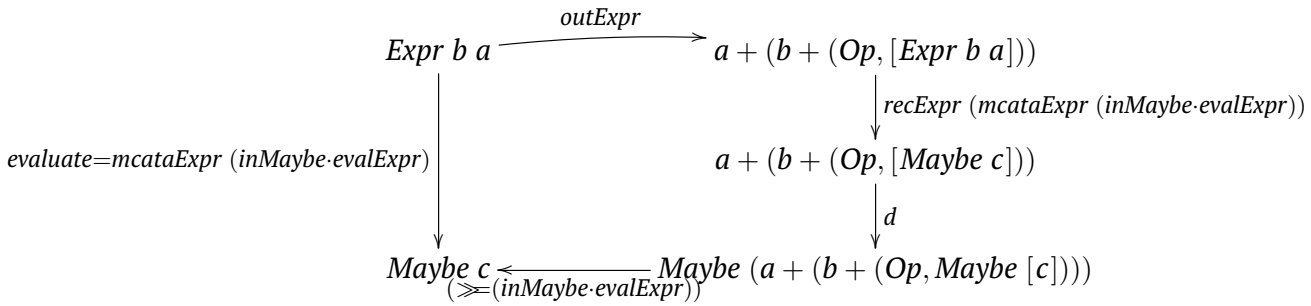
Avaliação de expressões:

Para resolver o problema 4.5, usamos o catamorfismo monádico definido anteriormente:

$$
\begin{aligned}
&evaluate = mcataExpr\ (inMaybe \cdot evalExpr)\\
&evalExpr :: (Num\ a, Ord\ a) \Rightarrow b + (a + (Op, Maybe\ [a])) \to () + a\\
&evalExpr = [\underline{i_1\ ()}, [i_2, \widehat{evalOp}]]\\
&evalOp :: (Num\ a, Ord\ a) \Rightarrow Op \to Maybe\ [a] \to () + a\\
&evalOp\ Suc\ (Just\ [x]) = i_2\ (x + 1)\\
&evalOp\ Add\ (Just\ [x, y]) = i_2\ (x + y)\\
&evalOp\ Mul\ (Just\ [x, y]) = i_2\ (x * y)\\
&evalOp\ ITE\ (Just\ [x, y, z]) = \mathbf{if}\ x \not\equiv 0\ \mathbf{then}\ i_2\ (y)\ \mathbf{else}\ i_2\ (z)\\
&evalOp\ _\ _ = i_1\ ()
\end{aligned}
$$

A função `evalExpr` retorna `i1()` se receber uma variável ou uma operação sem sentido e retorna o valor encapsulado em `i2`, se receber um número ou uma operação válida. A função `evalOp` é uma função auxiliar de `evalExpr`, que avalia operações.

Diagrama do evaluate:



Fim